

PlaylistBridge

Technical Documentation

v2.1 · June 2025 · playlistbridge.netlify.app

- No Login Required
- Vanilla JS / No Frameworks
- Firebase + Netlify
- PWA / Offline Support

PlaylistBridge is a free web application that converts plain-text song lists — typically generated by AI assistants such as ChatGPT, Claude, or Gemini — into instantly playable music links across Spotify, YouTube Music, and YouTube. No account, no login, no API keys required from the user.

Built by	Smit Patil (indie developer, age 17)
Stack	Vanilla JS · HTML · CSS · Firebase · Netlify
Deployment	Netlify (CDN) + Netlify Serverless Functions
Database	Firebase Firestore (community playlists, feedback)
Live URL	https://playlistbridge.netlify.app
Repo	PlaylistBridge--main (private)

Table of Contents

1. Core Concept & User Flow
2. Feature Reference
3. InstantPlay — In-Browser Player
4. Shareable Playlist Cards (Canvas)
5. Community Playlists

6.	Share System & Short Links
7.	Artist Mode
8.	Landing Pages & SEO
9.	Backend — Netlify Serverless Functions
10.	PWA & Service Worker
11.	Tech Stack & Architecture
12.	File Reference

1. Core Concept & User Flow

AI assistants generate playlists as plain text. PlaylistBridge closes the gap between that text and actual music playback — without requiring any account, login, or API credentials from the user.

User Flow

Step 1	User gets a text playlist from ChatGPT, Claude, Gemini, or any AI tool
Step 2	User pastes the list into PlaylistBridge's input box
Step 3	PlaylistBridge parses and cleans the song list (removes numbers, bullets, markdown)
Step 4	User selects target platform: Spotify, YouTube Music, or YouTube
Step 5	Per-song search links are generated instantly
Step 6	User clicks a song card to open it, or uses InstantPlay to listen in-browser
Step 7	(Optional) Share the playlist via a short URL or generate a shareable image card

2. Feature Reference

AI Playlist Parsing

Core

Automatically cleans AI-generated text playlists. Strips numbered lists, bullet points, markdown, parenthetical notes, and extra whitespace. Extracts clean "Artist — Title" pairs regardless of formatting.

- Handles ChatGPT, Claude, Gemini, and Grok playlist formats
- Smart parsing — does not break on unconventional formatting
- Regex-based cleaning pipeline in script.js

Multi-Platform Link Generation

Core

Generates direct search links for Spotify, YouTube Music, and YouTube. Platform switching is live — results update immediately without re-parsing.

- Platform-specific URL encoding for each service
- Theme switches to match each platform's color identity
- Bulk actions: Copy All links, Open All in tabs

InstantPlay — In-Browser Playback

Featured

Full music player built inside the browser using the YouTube IFrame API. Songs play without leaving the page. No YouTube tab required.

- Mini bar and full-screen expanded view (YouTube Music mobile style)
- Shuffle, previous, next, progress scrubbing
- Audio/video mode toggle
- Queue view with full track list
- 3-layer caching: in-memory → sessionStorage → Firestore
- Invidious API primary, YouTube HTML scrape as fallback

Shareable Playlist Cards

Featured

Canvas-based image generator that produces a shareable 1080px-wide card showing up to 5 songs with album art collage, blurred background, and branding.

- Instagram 4:5 aspect ratio — optimised for Stories and Feed
- Full-bleed blurred album art background
- Purple radial glow accents
- PlaylistBridge watermark for passive traffic
- Download as PNG — one click

Community Playlists

Social

Browse and discover playlists submitted by other users. Playlists are stored in Firebase Firestore and paginated for performance.

- Platform filter: All / Spotify / YouTube Music / YouTube
- Pagination with load-more
- Preview before loading
- Direct short link to each community playlist

Share System & Short Links

Social

Any generated playlist can be saved and shared as a short URL (e.g. `playlistbridge.netlify.app/p/abcd1234`). The Netlify share function handles storage in Firestore.

- One-click share button on result screen
- URL encodes platform selection
- Shared playlists load instantly — no re-parsing required
- Session hash system restores state on back navigation

Artist Mode

Detection

Single-line inputs that look like artist names (not song titles) are automatically detected and routed to an artist search page instead of song-by-song conversion.

- Heuristic detection: word count, dash separators, numbering patterns
- Generates artist search link on selected platform
- Displays artist name prominently

Autocomplete / Suggestions

UX

As the user types, a suggestion box appears with quick-fill options based on common AI playlist formats.

- Debounced input listener
- Keyboard navigation (arrow keys + Enter)
- ESC to dismiss

Example Playlist Templates

UX

Quick-start chips that load pre-built sample playlists for new users.

- Pop Hits, 90s Classics, AI Sample playlist
- One click to populate the input field
- Helps users understand the expected format

Feedback System

Ops

In-page star rating and text feedback form. Submissions are stored via a Netlify serverless function to Firebase Firestore.

- Star rating (1–5)
- Optional text comment
- Netlify function: `feedback.js`

PWA — Progressive Web App

Platform

PlaylistBridge is installable as a PWA on Android and desktop. The service worker pre-caches the app shell for offline access.

- App shell cached on install (Cache First strategy)
- Netlify functions: Network First with graceful fallback
- External services (Firebase, YouTube): Network Only
- `site.webmanifest` with icons at 192px and 512px

Odesli / Song.link Integration

API

When a user pastes a music platform URL (e.g. a Spotify track link), the Odesli API is called server-side to resolve cross-platform equivalents.

- Proxied via Netlify function to avoid CORS issues
- Returns links for all major streaming platforms
- Graceful error handling if lookup fails

3. InstantPlay — In-Browser Player

InstantPlay is a custom full-screen music player built on top of the YouTube IFrame API. It resolves YouTube video IDs for each track on demand and plays them inside the page — no separate YouTube tab required.

Video ID Resolution Pipeline

- 1 Check in-memory JS cache (same session, instant)
- 2 Check sessionStorage cache (survives page refresh)
- 3 Check Firebase Firestore cache (shared across users)
- 4 Call Netlify function: ytsearch.js
- 5 ytsearch.js tries multiple Invidious API instances
- 6 If all Invidious instances fail: scrape YouTube HTML for videoid
- 7 Cache result at all three layers for future use

Player Modes

The player has two visual states: a collapsed mini-bar that docks at the bottom of the screen, and a full-screen expanded view that mirrors the YouTube Music mobile UI.

Mode	Description	Controls Available
Mini Bar	Compact bottom dock — always visible during playback	Play/pause, prev, next, progress bar, shuffle, close
Expanded View	Full-screen overlay with artwork, queue, and all controls	All mini controls + audio/video toggle, queue list, seek

4. Shareable Playlist Cards (Canvas)

sharecard.js generates a 1080 x variable-height PNG image of the current playlist using the HTML5 Canvas API. The card is designed for Instagram (4:5 ratio) but works across all social platforms.

Property	Value
Canvas width	1080 px
Max songs shown	5 tracks
Background	Full-bleed blurred album art + dark gradient overlay
Header	Album art collage (up to 5 thumbnails)
Glow effects	Purple radial gradient — top right
Song list	Track thumbnail + title + artist name
Branding	PlaylistBridge logo + URL watermark at bottom
Export	PNG download via canvas.toDataURL()

5. Community Playlists

community.html is a discovery page that surfaces playlists shared by other users. Data is stored in Firebase Firestore and fetched client-side with pagination.

- Platform filter bar: All, Spotify, YouTube Music, YouTube
- Paginated results — load more on demand
- Each card shows: playlist name, song count, platform badge, short link
- Preview panel — click a playlist to see its track list before loading
- Direct link to open any community playlist in the main app

6. Share System & Short Links

share-system.js handles the full share flow. Clicking the share button saves the current playlist to Firestore via the Netlify share function and returns a short URL the user can copy and send.

Short URL format:

```
https://playlistbridge.netlify.app/p/{playlistId}
```

Session Hash System

The app encodes the current playlist + platform in the URL hash on every state change. This means the browser back button and refresh correctly restore the previous state without a round-trip to Firestore.

7. Artist Mode

When a single-line input is detected as an artist name rather than a playlist, the app switches to Artist Mode — displaying an artist search button instead of song cards.

Detection heuristics (isArtistInput function in script.js):

- Input must be exactly one line
- Word count must be 4 or fewer
- Must not contain dash (–), 'feat.', 'ft.', 'official', 'lyric', 'video'
- Must not start with a numbered list prefix (e.g. '1.' or '1 -')

8. Landing Pages & SEO

PlaylistBridge includes multiple SEO-optimised landing pages targeting specific AI + music platform search queries. All pages include structured data (FAQPage, HowTo schema) and direct answer blocks.

File	Target Query
chatgpt-to-spotify.html	ChatGPT playlist to Spotify
claude-to-spotify.html	Claude playlist to Spotify
gemini-playlist-player.html	Gemini playlist player
grok-playlist-player.html	Grok playlist player
text-to-playlist.html	Text to playlist converter
text-to-youtube-music.html	Text to YouTube Music
paste-ai-playlist.html	Paste AI playlist
how-to-play-chatgpt-playlist.html	How to play ChatGPT playlist

9. Backend — Netlify Serverless Functions

Function	Purpose
ytsearch.js	YouTube video ID resolution. Tries multiple Invidious instances, falls back to YouTube HTML scrape.
odesli.js	Proxies requests to the Odesli (song.link) API to resolve cross-platform music links.
share.js	Saves playlist data to Firebase Firestore and returns a generated short playlist ID.
feedback.js	Receives star rating + text feedback and writes it to Firestore.
itunes.js	iTunes Search API proxy — used for track metadata and artwork lookup.
instagram.js	Instagram Reel audio detection helper.

<code>reelSong.js</code>	Identifies the song in an Instagram Reel URL.
--------------------------	---

10. PWA & Service Worker

sw.js implements a service worker that makes PlaylistBridge installable and partially functional offline.

Resource Type	Caching Strategy	Rationale
App shell (HTML/CSS/JS/icons)	Cache First	Static assets — always serve fast
Netlify Functions (API)	Network First	Dynamic data — fallback gracefully if offline
Firebase / YouTube	Network Only	Real-time data — never cache

11. Tech Stack & Architecture

Layer	Technology	Notes
Frontend	Vanilla JS / HTML / CSS	Zero frameworks — intentional design choice
Player	YouTube IFrame API	Embedded in player.js
Image Gen	HTML5 Canvas API	sharecard.js — no external lib
Backend	Netlify Serverless Functions	Node.js — JS modules (ESM)
Database	Firebase Firestore	Community playlists, share links, feedback
CDN / Hosting	Netlify	Auto-deploy from Git
PWA	Service Worker (sw.js)	Cache-first shell, manifest.webmanifest
Search	Invidious API + YT scrape	No Google API key required
Song Links	Odesli (song.link) API	Cross-platform URL resolution

12. File Reference

File	Purpose
<code>index.html</code>	Main app page — playlist input, results, InstantPlay trigger
<code>script.js</code>	Core logic: parsing, platform links, artist detection, session hash
<code>ui.js</code>	UI rendering: track cards, platform themes, bulk actions
<code>player.js</code>	InstantPlay — full-screen YouTube IFrame player (v4.0)

sharecard.js	Canvas-based shareable playlist image generator (v2.0)
share-system.js	Share button, short link generation, Firestore write
os.js	OS-level utilities (clipboard, open-all, misc helpers)
style.css	Global styles — dark theme, platform color vars
sw.js	Service worker — PWA caching strategies
community.html	Community playlists discovery page
developer.html	Developer portfolio page (animated, particle canvas)
netlify/functions/ytsearch.js	YouTube video ID resolver (Invidious + scrape fallback)
netlify/functions/share.js	Playlist save-to-Firestore + short ID generation
netlify/functions/feedback.js	Feedback form submission handler
netlify/functions/odesli.js	Odesli API proxy (cross-platform song links)
netlify/functions/itunes.js	iTunes Search API proxy (metadata/artwork)
netlify/functions/instagram.js	Instagram Reel audio detection
netlify/functions/reelSong.js	Reel song identification
sitemap.xml	XML sitemap for all pages
robots.txt	Search engine crawl rules
llms.txt	Machine-readable description for AI crawlers
netlify.toml	Netlify build + function config
_redirects	Netlify URL rewrite rules (short link routing)